

Assignment 3

1. Write a program that finds all prime numbers between 1 and 100,000. Divide the range into N segments (where N is a command-line argument). Launch N threads using `thread_create()`, each responsible for one segment.
2. Write a program that prints its own Process ID (PID). Set an alarm for 5 seconds. If the user doesn't press "Ctrl+C" to exit before then, the program should send a `SIGKILL` to itself.
 - **Logic:**
 1. Get the current PID using `getpid()`.
 2. Set a signal handler for `SIGALRM` using `signal()`.
 3. Set an alarm for 5 seconds using `alarm(5)`.
 4. In the `SIGALRM` handler, use `kill()` to terminate the process.
3. Create a deeply nested mathematical function (e.g., main calls A, A calls B, B calls C). If function C detects a "division by zero" error, it should jump directly back to main without returning through B and A.
 - **Logic:**
 1. Define a global `jmp_buf`.
 2. In main, use `setjmp()` to save the stack environment.
 3. If `setjmp` returns 0, proceed to call function A.
 4. In function C, if an error occurs, call `longjmp()`.
4. Build a program that asks for a user's name. The user has only 10 seconds to type it. If they take too long, the program should print "Timeout!" and exit. Use `read` and `write` instead of `scanf/printf`.
 - **Logic:**
 1. Use `write(1, ...)` to print the prompt to stdout (FD 1).
 2. Use `setjmp` to mark the "start" of the prompt.
 3. Set an `alarm(10)` and a `SIGALRM` handler.
 4. Inside the handler, use `longjmp` to return to the `setjmp` point with a "timeout" status.

System Call Reference Table

Function Header	Prototype / Syntax	Purpose
getpid	<unistd.h> pid_t getpid(void);	Get the ID of the calling process.
alarm	<unistd.h> unsigned int alarm(unsigned int seconds);	Arranges for a SIGALRM signal to be delivered.
kill	<signal.h> int kill(pid_t pid, int sig);	Send any signal to any process or process group.
signal	<signal.h> void (*signal(int sig, void (*func)(int)))(int);	Register a function to handle a specific signal.
read	<unistd.h> ssize_t read(int fd, void *buf, size_t count);	Read data from a file descriptor (0 for stdin).
write	<unistd.h> ssize_t write(int fd, const void *buf, size_t count);	Write data to a file descriptor (1 for stdout).
setjmp	<setjmp.h> int setjmp(jmp_buf env);	Save the current calling environment/stack frame.
longjmp	<setjmp.h> void longjmp(jmp_buf env, int val);	Restore the environment saved by setjmp.